UNIT 7 70 min

# Lists

Ordered collections: indexing, slicing, changing and growing a list, sorting, and the aliasing trap.

---

**By the end of this unit you will be able to**

- Create lists and access items by index, including negative indexes
- Slice a list and explain what the slice includes
- Add, remove, and change items with `append`, `insert`, `remove`, `pop`, and assignment
- Explain why a list is mutable and what aliasing means
- Loop over a list and build a new list from it

## What a list is

A **list** is an ordered collection of values, written in square brackets and separated by commas. The items can be any type, and they can be mixed. Lists are **mutable**: you can change them after they are created.

lists_basics.py

```
scores = [72, 95, 88, 61]
names = ["Ana", "Ben", "Cy"]
mixed = [1, "two", 3.0, True]

print(scores)
print(len(names))
print(type(mixed))
```

## Indexing and slicing

Indexes start at 0. `list[-1]` is the last item. A slice `list[start:stop]` returns a new list that includes `start` and excludes `stop`.

indexing.py

```
letters = ["a", "b", "c", "d", "e"]
print(letters[0])     # a
print(letters[-1])    # e
print(letters[1:4])   # ['b', 'c', 'd']
print(letters[:2])    # first two
print(letters[3:])    # from index 3 to the end
```

**Watch out**

Asking for an index that does not exist raises `IndexError: list index out of range`. For a list of 5 items, the valid indexes are 0 to 4 (or -5 to -1).

## Changing a list

| Action | Code | Effect |
|---|---|---|
| Add to the end | `items.append(x)` | Adds one item at the end |
| Insert at a position | `items.insert(0, x)` | Puts x at index 0; everything shifts right |
| Remove by value | `items.remove(x)` | Removes the first match; error if absent |
| Remove by position | `items.pop(i)` | Removes and returns the item at i (default: last) |
| Replace | `items[2] = x` | Overwrites the item at index 2 |

changing.py

```
fruit = ["apple", "banana"]
fruit.append("cherry")
fruit.insert(0, "avocado")
fruit[1] = "blueberry"
removed = fruit.pop()
print(fruit)
print("removed:", removed)
```

Methods like `append` change the list in place and return `None`. Do not write `x = x.append(1)`. You would overwrite the list with `None`.

## Sorting

`sorted(items)` returns a new sorted list and leaves the original alone. `items.sort()` sorts in place and returns `None`. Add `reverse=True` for descending order.

## Aliasing: two names, one list

Assigning a list to a new name does **not** copy it. Both names point to the same list. Changing one changes the other. Use `.copy()` or `list(original)` when you need an independent copy.

**Trace: aliasing** TRACE

```
1  a = [1, 2]
2  b = a
3  b.append(3)
4  print(a)
```

| Step | Line run | Variables after the line | Output so far |
|---|---|---|---|
| 1 | 1 | `a = [1, 2]` | — |
| 2 | 2 | `a = [1, 2], b = [1, 2]` | — |
| 3 | 3 | `a = [1, 2, 3], b = [1, 2, 3]` | — |
| 4 | 4 | `a = [1, 2, 3], b = [1, 2, 3]` | `[1, 2, 3]` |

## Looping over a list

A `for` loop visits each item in order. Use `enumerate()` when you also need the index.

loop_lists.py

```
names = ["Ana", "Ben", "Cy"]
for name in names:
    print("Hi", name)

for i, name in enumerate(names):
    print(i, name)
```

**Tip**

Do not add or remove items from a list while looping over it. Build a new list instead: `evens = [n for n in nums if n % 2 == 0]` is the compact form of a loop with `append`.

## Predict the output THINK FIRST

Watch for in-place methods and aliasing.

```
nums = [10, 20, 30]
print(nums[1])
```

Answer: `20`

```
nums = [10, 20, 30]
print(nums[-1])
```

Answer: `30`

```
nums = [10, 20, 30]
nums.append(40)
print(len(nums))
```

Answer: `4`

```
print([1, 2, 3, 4, 5][1:4])
```

Answer: `[2, 3, 4]`

```
letters = ["a", "b"]
letters.insert(0, "z")
print(letters)
```

Answer: `['z', 'a', 'b']`

```
x = [3, 1, 2]
print(x.sort())
```

Answer: `None`
`sort()` changes the list and returns nothing.

```
print(5 in [1, 2, 3])
```

Answer: `False`

```
nums = [1, 2, 3]
nums[5] = 9
```

Answer: `IndexError`

## Practice

▼ **Exercise 1 — Average score**

Given `scores = [72, 95, 88, 61]`, print the average. Use `sum()` and `len()`.

▼ **Show solution**

Solution

```
scores = [72, 95, 88, 61]
average = sum(scores) / len(scores)
print(average)
```

▼ **Exercise 2 — Squares**

Build a list called `squares` containing the squares of 1 through 5 (1, 4, 9, 16, 25). Use a loop and `append`. Print the list.

▼ **Show solution**

**Solution**

```
squares = []
for n in range(1, 6):
    squares.append(n * n)
print(squares)
```

**Discussion**

1. Why do you think Python makes lists mutable instead of returning a new list for every change? What does that cost in clarity?
2. Predict: if two friends share one shopping list in a file and one of them deletes an item, what happens to the other? How is that like aliasing?
3. When is `sorted()` the better choice than `sort()`?

▼ **Facilitator notes**

The aliasing trace is the most important picture in this unit. Have students predict the output of `b = a.copy()` followed by `b.append(3)` before running it.

## Key terms

**List**

An ordered, mutable collection of values written in square brackets.

**Index**

The position of an item in a list, starting at 0. Negative indexes count from the end.

**Slice**

A new list made from a range of positions, written `items[start:stop]`.

**Mutable**

A value that can be changed after it is created. Lists and dictionaries are mutable.

**Method**

A function attached to an object. `append`, `insert`, `pop`, and `sort` are list methods.

**Aliasing**

When two names refer to the same object, so a change through one name is visible through the other.

**enumerate()**

A built-in that gives each item of a sequence together with its index, for use in a loop.

**IndexError**

The error raised when an index is outside the range of a list.

**List comprehension**

A compact expression that builds a list from a loop, such as `[n * n for n in range(5)]`.

See all terms in the glossary →